LeNet-5 卷积神经网络 MNIST 手写字符识别实验

项目概述

本项目基于 **MindSpore** 实现经典的 **LeNet-5** 卷积神经网络，用于识别 MNIST 手写数字数据集。实验采用控制变量法，对 **学习率（Learning Rate）**、**批大小（Batch Size）** 和 **优化器（Optimizer）** 三组关键超参数进行对比，分析不同设置对模型训练效果的影响。

实验目的

本实验旨在通过构建经典的 LeNet-5 卷积神经网络，实现对 MNIST 手写数字数据集的识别。在此基础上，采用控制变量法分别调整学习率、批大小和优化器三种关键超参数，探究不同参数设置对模型训练过程及最终识别率的影响。

文件结构

```
lenet5_experiment/
├── readme.md                                # 当前项目说明与实验报告
├── data/                                    # 数据集目录
│   └── MNIST/                               # MNIST 原始数据
├── download_mnist.py                        # 下载 MNIST 数据集的脚本
├── environment.yml                          # 项目环境配置文件
├── experiment_pytorch.py                    # PyTorch 版本的对比实验脚本
├── experiment_results.json                  # 参数对比实验结果数据
├── experiment_rmsprop_runner_v2.json        # 单独rmsprop模块运行结果(lr=0.001)
├── experiment_runner_v2.py                  # MindSpore 版本参数对比实验
├── experiment_runner_v2_rmsprop.py          # 单独的rmsprop模块
主脚本
├── figures/                                 # 实验结果图表目录
│   ├── batch_size_comparison.png            # 批大小对比图
│   ├── comprehensive_comparison.png         # 综合对比图
│   ├── learning_rate_comparison.png         # 学习率对比图
│   ├──  optimizer_comparison.png             # 优化器对比图
│   └── rmsprop_runner_v2_comparison.png     # 不同学习率rmsprop的对比度
├── lenet5_base.py                           # LeNet-5 网络定义与基础训练代码
├── log/                                     # 训练与可视化日志目录
│   ├── torch.log                            # PyTorch 实验日志
│   ├── trainv2.log                          # MindSpore 参数实验日志
│   ├──  visualize.log                        # 图表生成日志
│   ├── rmsprop_lr0001.log
│   └──rmsprop_lr001.log
└── visualize_results.py                     # 实验结果可视化脚本
```

核心代码说明

1. `lenet5_base.py`

功能： 定义 LeNet-5 网络结构，并实现基础的数据加载、模型训练和评估功能。

主要类和函数：

- `LeNet5(nn.Cell)`：LeNet-5 网络类，包含卷积层、池化层和全连接层。
- `create_mnist_dataset()`：创建 MNIST 数据集加载器。
- `train_model()`：模型训练函数。
- `evaluate_model()`：模型评估函数。

使用方法：

2. `experiment_runner_v2.py`

功能： 使用 MindSpore 框架运行参数对比实验，包括学习率、批大小和优化器三组实验。

实验设计：

- **学习率对比**：固定 SGD 优化器和批大小 32，测试学习率 `0.001`、`0.01`、`0.1`
- **批大小对比**：固定 SGD 优化器和学习率 0.01，测试批大小 `16`、`32`、`64`
- **优化器对比**：固定学习率 0.01 和批大小 32，测试 `SGD`、`Adam`、`RMSprop`

输出： `experiment_results.json` 文件包含所有实验的详细结果。

3. `experiment_pytorch.py`

功能： 使用 PyTorch 框架实现相同的参数对比实验，作为对照或备选实现。

4. `visualize_results.py`

功能： 读取实验结果 JSON 文件并生成可视化图表。

生成图表：

1. `learning_rate_comparison.png`：学习率对比图
2. `batch_size_comparison.png`：批大小对比图
3. `optimizer_comparison.png`：优化器对比图
4. `comprehensive_comparison.png`：综合对比图
5. `rmsprop_runner_v2_comparison.png`：rmsprop在lr=0.001和0.01的对比图

网络结构与实验设置

1. LeNet-5 网络结构

LeNet-5 是经典的卷积神经网络结构之一，适合用于手写数字识别任务。当前实验脚本中的 PyTorch 版本与 MindSpore `runner_v2` 已统一为先将 MNIST 图像缩放到 `32×32`，再输入网络，因此实际使用的结构如下：

层级	类型	参数设置	输出尺寸
Input	输入层	单通道灰度图	1×32×32
C1	卷积层	6 个 5×5 卷积核，步长 1，无填充	6×28×28
S2	池化层	2×2 平均池化，步长 2	6×14×14
C3	卷积层	16 个 5×5 卷积核，步长 1，无填充	16×10×10
S4	池化层	2×2 平均池化，步长 2	16×5×5
Flatten	展平层	将特征图展平	400
F5	全连接层	120 个神经元	120
F6	全连接层	84 个神经元	84
Output	输出层	10 个神经元，对应数字 0–9	10

从当前 `experiment_results.json` 的结果看，这个 `32×32` 版本的 LeNet-5 在 20 个 epoch 内可以稳定收敛到较高精度，说明将输入放大到更接近经典 LeNet-5 设定后，卷积层和全连接层的尺寸衔接是合理的。

2. 实验环境与数据集

本实验当前的主要结果来自 **MindSpore** **`experiment_runner_v2.py`**，数据集使用经典的 **MNIST 手写数字数据集**。当前实验中统一采用以下预处理流程：

1. 固定随机种子方便复现试验
2. 将单通道灰度图像缩放到 `32×32`
3. 转换为浮点张量
4. 使用 `mean=0.1307`、`std=0.3081` 进行标准化
5. 采用单通道输入格式参与训练

根据当前脚本设置与 `experiment_results.json`，`runner_v2` 使用完整 MNIST 官方划分进行实验：训练集 `60000` 张、测试集 `10000` 张，每组参数训练 `20` 个 epoch。因此后续结果更适合解释为在完整 MNIST 基准上的对比结论。

3. 实验设计

本实验采用控制变量法，共设计三组对比实验。根据当前 `experiment_results.json` 的结果，三组实验都能够在 20 个 epoch 内拉开明显差异，因此当前设置足以比较不同超参数对收敛速度、稳定性和最终精度的影响。

1. **学习率对比实验**：固定优化器为 `SGD`，批大小为 `32`，比较学习率 `0.001`、`0.01` 和 `0.1`。
2. **批大小对比实验**：固定优化器为 `SGD`，学习率为 `0.01`，比较批大小 `16`、`32` 和 `64`。
3. **优化器对比实验**：固定学习率为 `0.01`、批大小为 `32`，比较 `SGD`、`Adam` 和 `RMSprop`。

从当前 `experiment_results.json` 的结果看，当前实验设置可以得到以下直接结论：

1. 学习率方面，`0.001` 和 `0.01` 的结果几乎持平，最终测试准确率分别为 `98.54%` 和 `98.55%`；`0.1` 虽然仍能收敛，但最终下降到 `97.25%`，说明在完整 MNIST 上过大的学习率会牺牲最终精度。

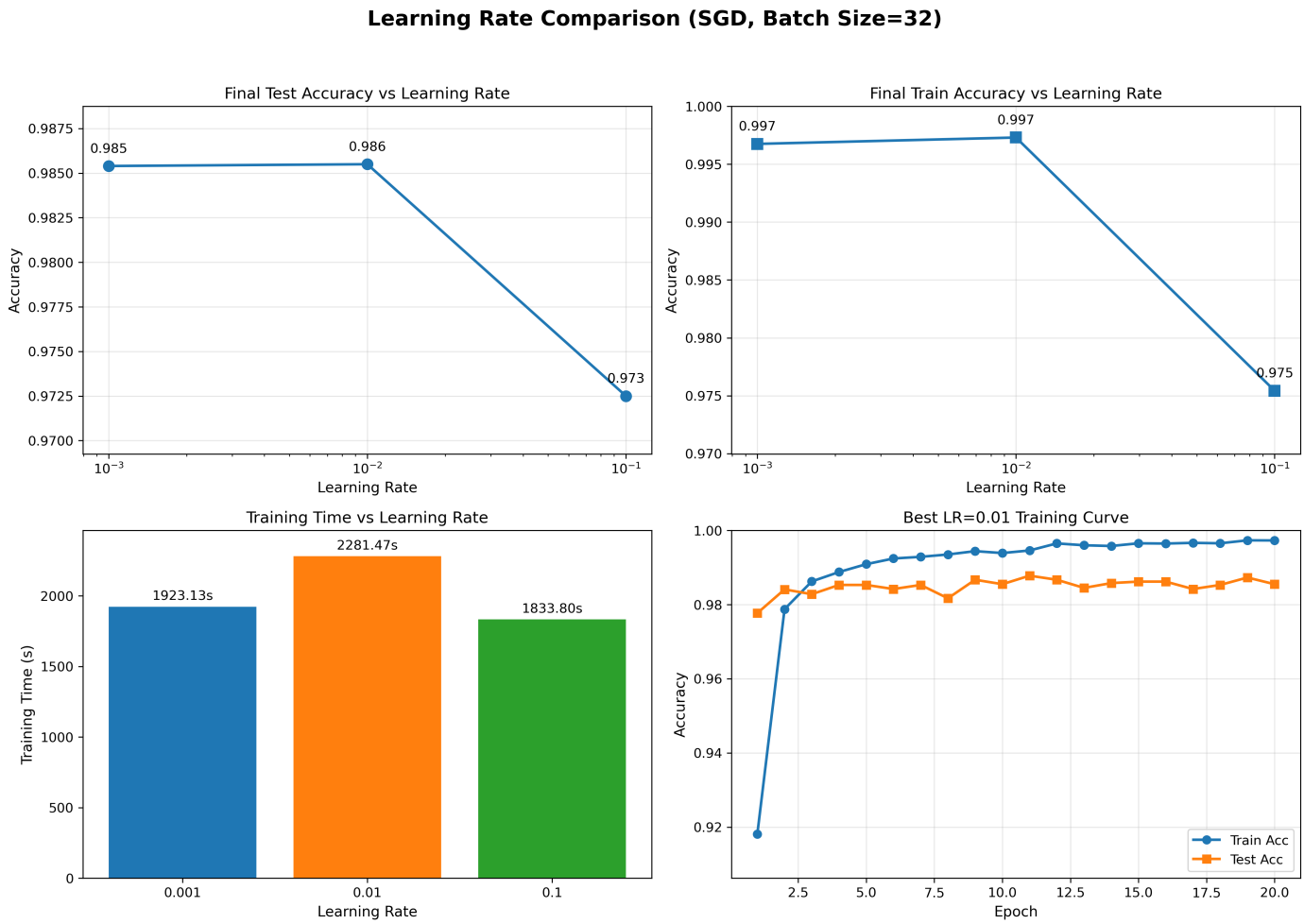
2. 批大小方面，`64` 同时取得了最高测试准确率 `98.87%` 和最短训练时间 `1121.34s`；`32` 次之；`16` 最慢，且准确率最低。这说明在当前 MindSpore PYNATIVE 配置下，较大的批大小更有利于效率和最终性能。
3. 优化器方面，`SGD` 表现最好，最终测试准确率 `98.55%`；`Adam` 最终只有 `92.50%`，且后期出现明显训练崩溃；`RMSprop` 直接发散，最终仅 `14.87%`。说明在当前实现与超参数设置下，自适应优化器对学习率更敏感，稳定性明显弱于 SGD。

实验结果与分析

1. 学习率对比实验

学习率	最终测试准确率	结果分析
0.001	98.54%	稳定收敛，最终结果与 0.01 基本持平
0.01	98.55%	本组最优，但相对 0.001 优势很小
0.1	97.25%	可以收敛，但最终精度明显低于前两者

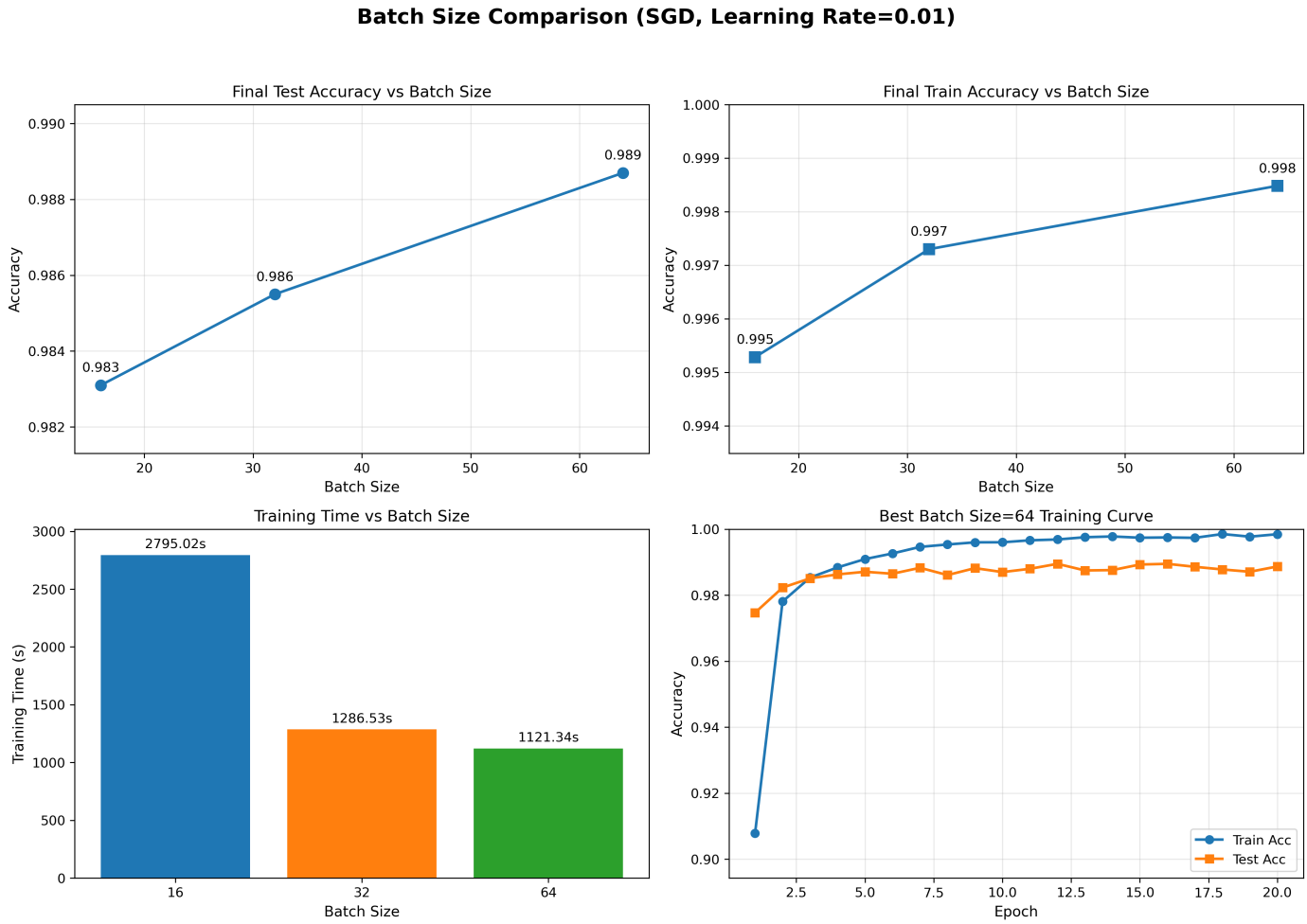
分析：
从当前结果来看，`0.001` 与 `0.01` 都是合适的学习率，二者最终准确率几乎一致，其中 `0.01` 仅以极小优势领先。`0.1` 虽然没有完全训练失败，但最终精度明显下降，说明在完整 MNIST 上过大的学习率会使优化过程更粗糙，难以达到最佳解。



2. 批大小对比实验

批大小	最终测试准确率	训练耗时 (s)	结果分析
16	98.31%	2795.02	最慢，且最终精度反而最低
32	98.55%	1286.53	精度与速度较均衡
64	98.87%	1121.34	当前实验中准确率最高，且耗时最短

分析：
在当前记录的实验结果中，批大小 `64` 的表现最好，不仅测试准确率最高，而且训练时间最短；`32` 的表现也较为均衡；`16` 虽然也能稳定收敛，但耗时明显更长且最终准确率略低。说明在当前 MindSpore PYNATIVE 配置和完整 MNIST 数据规模下，较大的批大小更有利于提高训练效率，并取得更好的最终结果。

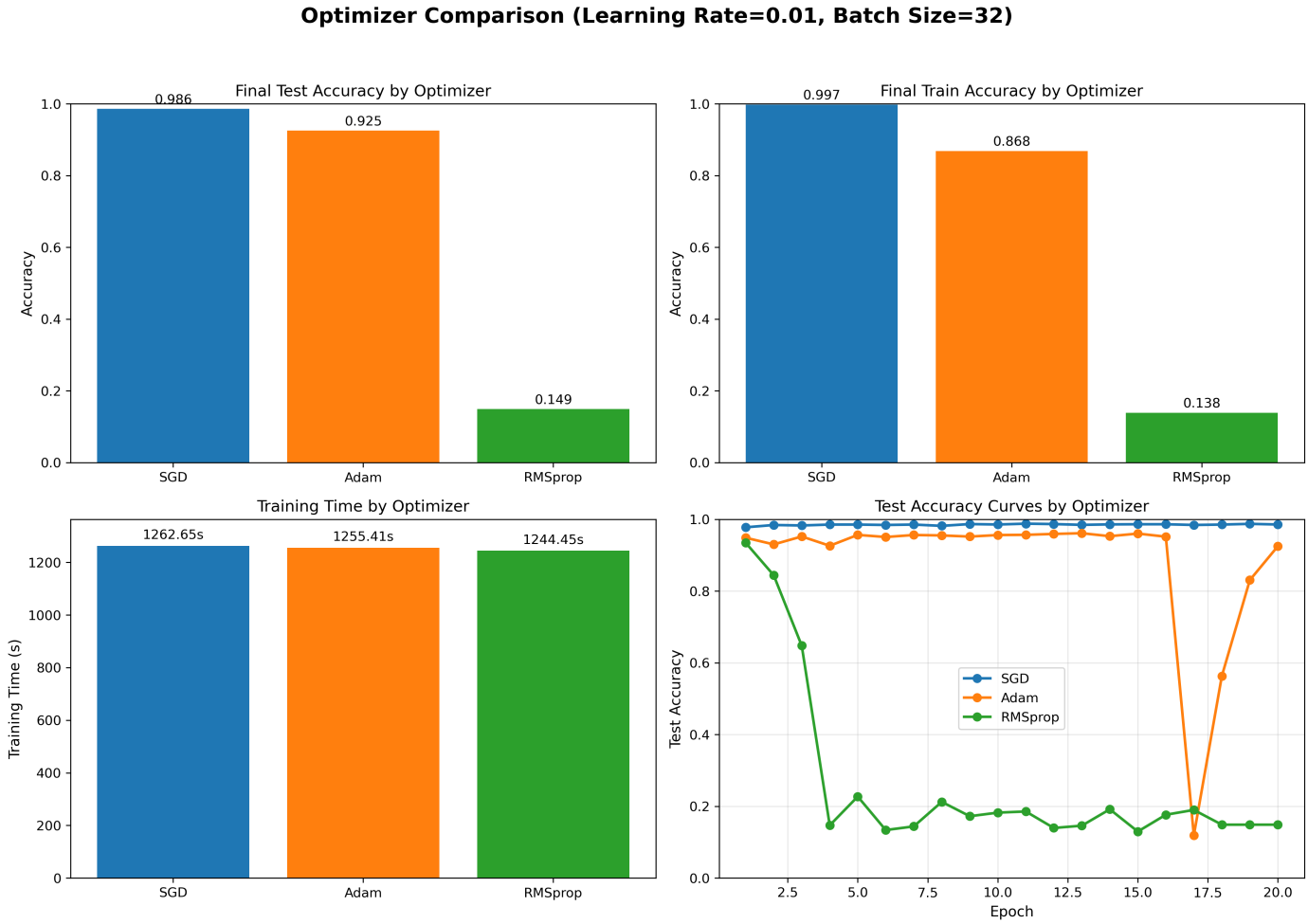


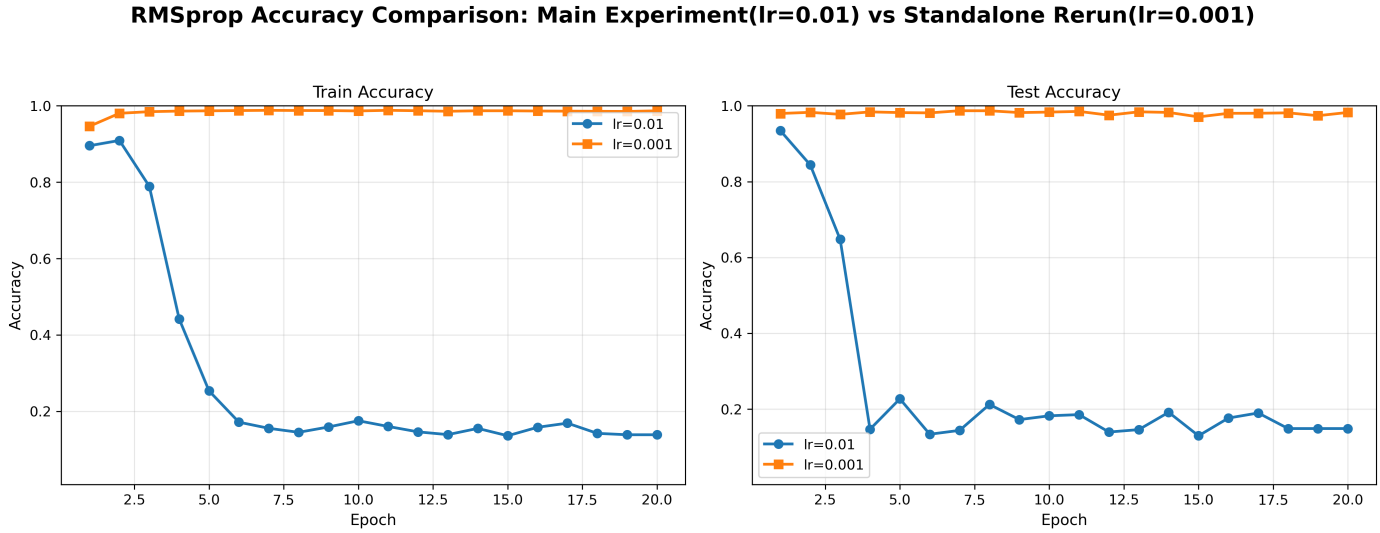
3. 优化器对比实验

优化器	最终测试准确率	结果分析
SGD	98.55%	当前实验中效果最好，且训练稳定
Adam	92.50%	前期可训练，但后期出现明显崩溃
RMSprop	14.87%	基本发散，训练与测试精度同时下跌

分析：

从当前结果来看，`SGD` 在该任务中的表现显著优于 `Adam` 和 `RMSprop`。`Adam` 在前期能够快速提高精度，但后期出现明显崩溃，导致最终结果回落到 `92.50%`。`RMSprop` 则更为极端，训练集和测试集精度同时快速下跌，核心原因是累积梯度机制下，`RMSprop` 更需要精细的学习率控制，我额外比较 `lr=0.01` 和 `lr=0.001` 条件下的准确度，可以发现在后者情况下 `RMSprop` 也能取得不错的结果。

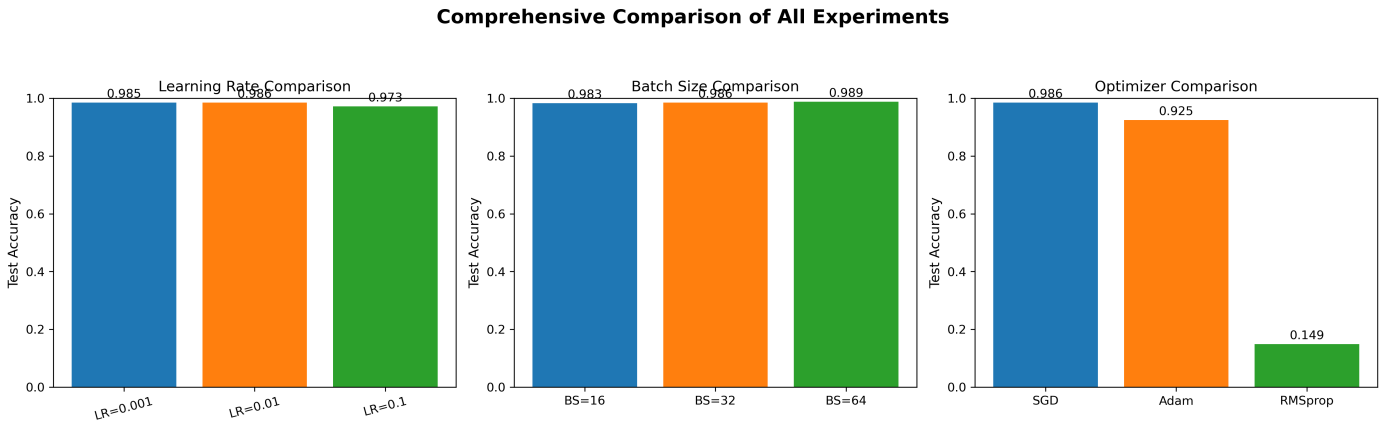




4. 综合结论

结合上述三组实验，当前这组 LeNet-5 + MNIST 实验结果可以概括为：

1. **学习率方面**：`0.001` 与 `0.01` 都能稳定达到接近 `98.5%` 的测试准确率，其中 `0.01` 略优；`0.1` 会拉低最终精度。
2. **批大小方面**：`64` 在当前实验中同时取得了最优准确率与最短训练时间，是最优设置。
3. **优化器方面**：`SGD` 的最终表现显著优于 `Adam` 和 `RMSprop`，后两者在当前学习率下稳定性不足。



5. PyTorch 架构对比

根据 `log/torch.log` 的结果，PyTorch 版本在相同的 `32×32` 输入、`20` 个 epoch 和完整 MNIST 设置下，整体表现出更强的优化器稳定性。学习率实验中，PyTorch 的 `SGD + lr=0.1` 最终达到 `99.12%`，高于 MindSpore `runner_v2` 中对应设置的 `97.25%`；优化器实验中，PyTorch 的 `SGD / Adam / RMSprop` 分别达到 `98.82% / 97.77% / 96.90%`，

另一方面，两者在批大小实验中的最优点并不完全一致。PyTorch 中 `batch_size=16` 略优，最终测试准确率 `98.84%`；MindSpore `runner_v2` 中则是 `batch_size=64` 最优，达到 `98.87%`，同时训练时间也最短，整体耗时约`1200s~1000s`。这表明两套框架即使网络结构一致，数据管线、优化器实现和训练后端的差异仍会影响最佳超参数区域。

结论

本实验成功实现了 LeNet-5 卷积神经网络在 MNIST 数据集上的训练与测试，并通过控制变量法分析了学习率、批大小和优化器对模型性能的影响。

根据当前 `experiment_results.json` 的实验记录，可以得出以下结论：

1. 学习率需要合理设置，在完整 MNIST 上 `0.001` 和 `0.01` 都表现稳定，而 `0.1` 会降低最终精度。
2. 在当前实验中，较大的批大小反而同时提升了速度和精度，`64` 是最佳设置。
3. 优化器的效果与实现和超参数密切相关，在当前 MindSpore `runner_v2` 设置下，`SGD` 明显优于 `Adam` 和 `RMSprop`。

最后

代码已在github开源[https://github.com/ChriCheng/lenet5]